Programação Estruturada em C

Aula 7 — Matrizes e Arranjos Paralelos

Prof. Me. Eng. Matheus Kotaki

A mudança desta aula

Até agora...

- vetores guardam vários valores
- funções organizam responsabilidades
- strings são vetores de char

A partir de hoje

- matrizes organizam dados em duas dimensões
- funções também recebem matrizes
- podemos armazenar várias strings
- o mesmo índice pode relacionar dados

A ideia central continua sendo organizar dados para processá-los com laços e funções.

Um problema antes da sintaxe

Precisamos registrar as notas de 3 alunos em 2 provas.

```
float p1[3];
float p2[3];
```

Funciona. Mas os dois vetores representam a mesma ideia: notas de alunos em avaliações.

Podemos usar uma única estrutura com linhas para alunos e colunas para provas.

Matriz: uma estrutura, duas dimensões

```
#define ALUNOS 3
#define PROVAS 2

float notas[ALUNOS][PROVAS];
```

- 3 linhas
- 2 colunas
- todos os elementos são float

	P1	P2
Aluno 0	`[0][0]`	`[0][1]`
Aluno 1	`[1][0]`	`[1][1]`
Aluno 2	`[2][0]`	`[2][1]`

Cada elemento é acessado com dois índices: [linha][coluna].

Índices: linha e coluna

notas[0][0]

Aluno 0 — P1

notas[0][1]

Aluno 0 — P2

notas[2][1]

Aluno 2 — P2

Para notas[3][2]:

linhas válidas → 0, 1 e 2
colunas válidas → 0 e 1

	P1	P2
Aluno 0	`[0][0]`	`[0][1]`
Aluno 1	`[1][0]`	`[1][1]`
Aluno 2	`[2][0]`	`[2][1]`

O índice começa em 0, assim como nos vetores.

Percorrendo uma matriz: laços aninhados

```
#define ALUNOS 3
#define PROVAS 2


for (int i = 0; i < ALUNOS; i++) {
    for (int j = 0; j < PROVAS; j++) {
        scanf("%f", &notas[i][j]);
    }
}
```

A cada repetição:

- `i` escolhe uma linha
- `j` percorre as colunas
- `notas[i][j]` acessa um elemento

	P1	P2
Aluno 0	7.0	8.5
Aluno 1	6.0	9.0
Aluno 2	8.0	7.5

Uma dimensão → um índice. Duas dimensões → dois índices.

Exibindo uma matriz

```
#define ALUNOS 3
#define PROVAS 2



for (int i = 0; i < ALUNOS; i++) {
    for (int j = 0; j < PROVAS; j++)
{
        printf("%.1f ", notas[i][j]);
    }
    printf("\n");
}
```

Exemplo de saída

```
7.0 8.5
6.0 9.0
8.0 7.5
```

Funções

```
#define LINHAS 2
#define COLUNAS 3

void lerMatriz(int m[][COLUNAS], int lin);
void exibirMatriz(int m[][COLUNAS], int lin);


int main() {
    int valores[LINHAS][COLUNAS];

    lerMatriz(valores, LINHAS);
    exibirMatriz(valores, LINHAS);
    return 0;
}
```

Para receber uma matriz, a função precisa conhecer o tamanho da segunda dimensão.

Função para ler a matriz

```
void lerMatriz(int m[][COLUNAS], int lin) {
    for (int i = 0; i < lin; i++) {
        for (int j = 0; j < COLUNAS; j++) {
            printf("Valor [%d][%d]: ", i, j);
            scanf("%d", &m[i][j]);
        }
    }
}
```

Função para exibir a matriz

```
void exibirMatriz(int m[][COLUNAS], int lin) {
    for (int i = 0; i < lin; i++) {
        for (int j = 0; j < COLUNAS; j++) {
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }
}
```

Prática 1

Um laboratório registrou temperaturas em 3 dias, nos períodos da manhã e da tarde.

- Use uma matriz 3 × 2.
- Crie uma função para ler as temperaturas.
- Crie uma função para exibir as temperaturas.
- Crie uma função que retorne a maior temperatura registrada.
- Organize o código com protótipos, main() e definições das funções.

Prática 1 — resolução: protótipos

```
#include <stdio.h>
#define DIAS 3
#define PERIODOS 2

/* Protótipos */
void lerTemperaturas(float t[][PERIODOS], int dias);

void exibirTemperaturas(float t[][PERIODOS], int dias);
float maiorTemperatura(float t[][PERIODOS], int dias);
```

Os protótipos ficam antes da main() e antecipam quais funções serão usadas.

Prática 1 — resolução: main()

```
int main() {
    float temperaturas[DIAS][PERIODOS];

    lerTemperaturas(temperaturas, DIAS);
    exibirTemperaturas(temperaturas, DIAS);

    printf("Maior: %.1f\n", maiorTemperatura(temperaturas, DIAS));

    return 0;
}
```

main() coordena as chamadas; cada função realiza uma tarefa específica.

Prática 1 — resolução: leitura

```
void lerTemperaturas(float t[][PERIODOS], int dias) {
    for (int i = 0; i < dias; i++) {
        for (int j = 0; j < PERIODOS; j++) {
            printf("Dia %d, periodo %d: ",i + 1, j + 1);
            scanf("%f", &t[i][j]);
        }
    }
}
```

Prática 1 — resolução: exibição

```
void exibirTemperaturas(float t[][PERIODOS], int dias) {
    for (int i = 0; i < dias; i++) {
        for (int j = 0; j < PERIODOS; j++) {
            printf("%.1f ", t[i][j]);
        }
        printf("\n");
    }
}
```

A função percorre toda a matriz e apresenta uma linha de cada vez

Prática 1 — resolução: maior temperatura

```
float maiorTemperatura(float t[][PERIODOS], int dias) {
    float maior = t[0][0];

    for (int i = 0; i < dias; i++) {
        for (int j = 0; j < PERIODOS; j++) {
            if (t[i][j] > maior) {
                maior = t[i][j];
            }
        }
    }

    return maior;
}
```

Prática 1 — exemplo de execução

```
Dia 1, periodo 1: 23.5
Dia 1, periodo 2: 28.0
Dia 2, periodo 1: 22.0
Dia 2, periodo 2: 30.5
Dia 3, periodo 1: 24.0
Dia 3, periodo 2: 27.0

23.5 28.0
22.0 30.5
24.0 27.0
Maior: 30.5
```

De uma string para várias strings

Uma string:

```
char nome[81];
```

Várias strings:

```
#define QTD 5
#define TAM_NOME 81

char nomes[QTD][TAM_NOME];
```

Como interpretar?

nomes[0] → uma string
nomes[1] → outra string
...
nomes[4] → outra string

Uma coleção de strings é uma aplicação muito útil de matriz de char

Funções com coleções de strings

```
#define QTD 3
#define TAM_NOME 51

/* Protótipos */
void lerNomes(char nomes[][TAM_NOME], int qtd);
void exibirNomes(char nomes[][TAM_NOME], int qtd);

int main() {
    char nomes[QTD][TAM_NOME];

    lerNomes(nomes, QTD);
    exibirNomes(nomes, QTD);
    return 0;
}
```

`char nomes[][TAM_NOME]`

Cada `nomes[i]` representa uma string completa.

Vetores paralelos: dados relacionados

Queremos armazenar, para cada aluno:

- RA
- nome
- notas
- faltas

O mesmo índice representa o mesmo aluno em todas as estruturas.

Estamos relacionando posições correspondentes

Notas em uma matriz dentro do conjunto de dados

```
#define QTD 5
#define TAM_NOME 81
#define QTD_NOTAS 2

int ras[QTD];
char nomes[QTD][TAM_NOME];
float notas[QTD][QTD_NOTAS];
int faltas[QTD];
```

`notas[i][0]`

P1 do aluno `i`

`notas[i][1]`

P2 do aluno `i`

Mesmo índice = mesmo aluno

índice	RA	Nome	P1	P2	Faltas
0	12345	Ana	7.0	8.0	2
1	34821	Bruno	5.5	7.0	6
2	52903	Carla	9.0	8.5	1

`ras[1]`, `nomes[1]`, `notas[1][0]`, `notas[1][1]` e `faltas[1]` pertencem ao Bruno

Dados armazenados × informações calculadas

Armazenamos

RA
nome
P1 e P2
faltas

Calculamos quando necessário

média
frequência
situação final

Nem toda informação precisa ocupar um vetor próprio

Média, frequência e aprovação

```
media = (notas[i][0] + notas[i][1]) / 2;

frequencia = (aulasDadas - faltas[i]) * 100 / aulasDadas;

if (media >= 6 && frequencia >= 75) {
    printf("Aprovado");
} else {
    printf("Reprovado");
}
```

Média

≥ 6,0

Frequência

≥ 75%

As duas condições precisam ser verdadeiras.

Prática 2

Desenvolva um programa que armazene os dados de 5 alunos.

- RA e nome em vetores paralelos.
- P1 e P2 em uma única matriz de notas.
- Quantidade de faltas em um vetor.
- Use fgets() para o nome e **retire o \n armazenado**.
- Antes de calcular os resultados, o usuário informa quantas aulas foram dadas.

- Calcule a média de cada aluno.
- Calcule a frequência de cada aluno.
- Apresente RA, nome, P1, P2, média e frequência.
- Informe “Aprovado” apenas se média ≥ 6 e frequência ≥ 75%.
- Organize a solução em funções, evite concentrar a lógica em main().

Não é necessário armazenar média, frequência ou situação em novos vetores

Prática 2 — resolução: protótipos

```
#include <stdio.h>
#include <string.h>
#define QTD 5
#define TAM_NOME 81
#define QTD_NOTAS 2

void lerAlunos(int ras[], char nomes[][TAM_NOME],
               float notas[][QTD_NOTAS], int faltas[], int qtd);

void exibirResultados(int ras[], char nomes[][TAM_NOME],
                      float notas[][QTD_NOTAS], int faltas[],
                      int aulasDadas, int qtd);
```

Prática 2 — resolução: main()

```
int main()
{
    int ras[QTD];
    char nomes[QTD][TAM_NOME];
    float notas[QTD][QTD_NOTAS];
    int faltas[QTD];
    int aulasDadas;

    lerAlunos(ras, nomes, notas, faltas, QTD);

    printf("Quantidade de aulas dadas: ");
    scanf("%d", &aulasDadas);

    exibirResultados(ras, nomes, notas, faltas, aulasDadas, QTD);

    return 0;
}
```

```
void lerAlunos(int ras[], char nomes[][TAM_NOME], float notas[][QTD_NOTAS],
               int faltas[], int qtd) {
    int c;
    for (int i = 0; i < qtd; i++) {
        printf("RA: ");
        scanf("%d", &ras[i]);
        while ((c = getchar()) != '\n' && c != EOF);

        printf("Nome: ");
        fgets(nomes[i], TAM_NOME, stdin);
        nomes[i][strcspn(nomes[i], "\n")] = '\0';

        printf("P1: ");
        scanf("%f", &notas[i][0]);
        printf("P2: ");
        scanf("%f", &notas[i][1]);
        printf("Faltas: ");
        scanf("%d", &faltas[i]);
    }
}
```

```
void exibirResultados(int ras[], char nomes[][TAM_NOME],
                      float notas[][QTD_NOTAS], int faltas[],
                      int aulasDadas, int qtd) {

    for (int i = 0; i < qtd; i++) {

        float media = (notas[i][0] + notas[i][1]) / 2.0;
        float frequencia = (aulasDadas - faltas[i]) * 100.0/ aulasDadas;

        printf("\nRA: %d | Nome: %s\n", ras[i], nomes[i]);
        printf("P1: %.1f | P2: %.1f\n", notas[i][0], notas[i][1]);
        printf("Media: %.1f | Frequencia: %.1f%%\n", media, frequencia);

        if (media >= 6.0 && frequencia >= 75.0)
            printf("Situacao: Aprovado\n");
        else
            printf("Situacao: Reprovado\n");
    }
}
```

Prática 2 — exemplo de resultado

```
Quantidade de aulas dadas: 40

RA: 123 | Nome: Ana Souza
P1: 7.0 | P2: 8.0
Media: 7.5 | Frequencia: 95.0%
Situacao: Aprovado

RA: 348 | Nome: Bruno Lima
P1: 5.5 | P2: 7.0
Media: 6.2 | Frequencia: 72.5%
Situacao: Reprovado
```